

Catertech Project Structure, Backend Flow, Security, and Neon/Drizzle Notes

This document explains the current Catertech website structure in simple full-stack terms. It is written for developers who are familiar with older separated projects such as:

```
backend/  
  config/  
  controllers/  
  middlewares/  
  models/  
  routes/  
  utils/  
  
frontend/  
  src/  
    app/  
    components/  
    redux/  
    api/
```

Catertech uses a different but professional structure because it is a Next.js App Router full-stack project. In this architecture, the frontend pages, backend API routes, server-side database queries, admin panel, and shared business logic live in one Next.js app.

High-Level Verdict

The current Catertech folder structure is suitable for a production-level Next.js application.

It should not be forced into a separate `backend/` and `frontend/` structure just because older Express/MongoDB projects were organized that way. Next.js App Router is designed to support backend and frontend code inside the same project while keeping clear boundaries:

Public website pages	-> <code>src/app/(site)</code>
Admin panel pages	-> <code>src/app/admin</code>
Backend API routes	-> <code>src/app/api</code>
Database connection/schema	-> <code>src/db</code>
Shared business logic	-> <code>src/lib</code>
Reusable UI components	-> <code>src/components</code>

The main improvement for future growth is not a full folder rewrite. The better improvement is to move heavier business logic out of large API route files into `service/repository` files, for example:

```
src/server/  
  services/  
  repositories/  
  validators/
```

That would make the code feel closer to `controllers`, `models`, and `services` in an Express project while still matching Next.js conventions.

Current Folder Structure

```
catertech_site/  
|  
|-- src/  
|   |  
|   |-- app/  
|   |   |
```

```

|-- (site)/
|   Public website pages.
|   Examples: homepage, shop, contact, trade enquiry, RFQ, services, blog.
|
|-- admin/
|   Admin panel pages.
|   Examples: dashboard, products, users, contacts, enquiries, RFQ, quotations.
|
|-- api/
|   Backend API routes.
|   Used for forms, login/logout, admin CRUD, uploads, quotes, RFQs, enquiries.
|
|-- auth/
|   Public auth UI.
|
|-- studio/
|   Sanity Studio route.
|
|-- db/
|   |
|   |-- index.ts
|   |   Creates the Drizzle client using Neon serverless connection.
|   |
|   |-- schema/
|   |   Drizzle database table definitions.
|   |   Similar to "models" in an Express/MongoDB project.
|   |
|   |-- migrations/
|   |   SQL migration files for database changes.
|   |
|-- components/
|   |
|   |-- admin/
|   |   Admin tables, forms, panels, product controls, dashboard components.
|   |
|   |-- shop/
|   |   Product cards, catalogue UI, cart UI, product detail UI.
|   |
|   |-- sections/
|   |   Public website sections used across pages.
|   |
|   |-- layout/
|   |   Header, footer, navigation, layout-level components.
|   |
|   |-- blog/
|   |   Blog-related components.
|   |
|   |-- legal/
|   |   Legal page components.
|   |
|   |-- ui/
|   |   Shared UI primitives.
|   |
|-- lib/
|   Shared application logic.
|   Examples: auth helpers, product mapping, catalogue queries, upload helpers,
|   email helpers, pricing helpers, validation helpers, cart storage.
|
|-- assets/
|   Imported images and static assets used by Next.js.
|
|-- sanity/
|   Sanity CMS setup.
|
|-- types/
|   Shared TypeScript types.
|
|-- public/
|   Static files served directly by the browser.
|
|-- docs/
|   Project documentation.
|
|-- scripts/
|   Utility scripts.
|
|-- middleware.ts
|   Request middleware for auth protection, admin protection, rate limiting,
|   same-origin checks, and security headers.

```

```

|
|-- drizzle.config.ts
|   Drizzle Kit configuration.
|
|-- next.config.ts
|   Next.js configuration.
|
|-- package.json
|   Dependencies and npm scripts.
|
|-- .env.local
|   Local environment variables and secrets.
|
|-- .env.example
|   Example environment variables.

```

Express/MongoDB Comparison

Old Express/MongoDB project	Catertech Next.js project
backend/config/	src/db/index.ts drizzle.config.ts .env.local
backend/models/	src/db/schema/index.ts
backend/controllers/	src/app/api/**/route.ts src/lib/*.ts
backend/routes/	src/app/api/**/route.ts
backend/middlewares/	middleware.ts src/lib/auth-user.ts src/lib/admin-roles.ts src/lib/user-auth-session.ts src/lib/security.ts
backend/utils/	src/lib/*.ts
frontend/src/app/	src/app/(site) src/app/admin
frontend/src/components/	src/components
frontend/src/redux/	No Redux folder currently. Uses React state/context where needed.
frontend/src/api/	Client components use fetch("/api/...") only where browser/API interaction is needed.

Main Backend Files and Use Cases

Database Layer

src/db/index.ts

Creates the Neon + Drizzle database client.

Flow:

DATABASE_URL -> Neon serverless client -> Drizzle ORM -> application queries

src/db/schema/index.ts

Defines database tables such as:

products

```
product_categories
product_subcategories
product_title_presets
users
auth_otp_challenges
contact_messages
trade_enquiries
rfq/event_submissions
quote/quotation_records
```

This is similar to `models/` in an Express/MongoDB project, but because Drizzle is SQL-first, these are typed PostgreSQL table definitions rather than Mongoose schemas.

API Layer

```
src/app/api
```

This is the backend API folder.

Each `route.ts` file is like an Express route/controller file. It can export functions such as:

```
export async function GET() {}
export async function POST(request: Request) {}
export async function PUT(request: Request) {}
export async function PATCH(request: Request) {}
export async function DELETE(request: Request) {}
```

Important API groups:

```
src/app/api/contact
  Saves public contact form submissions.
```

```
src/app/api/enquiry
  Saves trade enquiry submissions.
```

```
src/app/api/rfq
  Saves RFQ/event request submissions.
```

```
src/app/api/quote
  Saves quote/cart submissions.
```

```
src/app/api/chatbot-leads
  Saves WhatsApp/chatbot leads.
```

```
src/app/api/upload
  Handles admin media uploads.
```

```
src/app/api/auth
  Handles login, logout, session check, signup OTP send/verify.
```

```
src/app/api/admin
  Handles admin-only backend actions:
  products, categories, subcategories, product presets, stats, users,
  contacts, enquiries, RFQs, and quotations.
```

API Call Vs Direct Drizzle Query

Catertech uses two backend access patterns.

Pattern 1: Browser Calls API Route

Used when the browser must submit data, perform a private admin action, upload a file, log in, or update/delete something.

Example flow:

```
Browser/client component
  |
  | fetch("/api/contact")
  v
Next.js API route
  |
  | Drizzle query
  v
Neon PostgreSQL
```

Used for:

Contact form
Trade enquiry form
RFQ form
Quote/cart submission
Chatbot/WhatsApp leads
Login/logout/signup OTP
Admin create/update/delete product
Admin users
Admin product categories
Admin product presets
Uploads

Why API calls are correct here:

The browser is sending user-specific or private data.
The request needs validation.
The request may mutate the database.
The request may require admin authentication.
The browser must not access database credentials directly.

Pattern 2: Next.js Server Page Queries Drizzle Directly

Used when a public page only needs to display database content.

Important Catertech flow:

Next.js server page -> Drizzle -> Neon -> cached rendered shop page

Example:

```
src/app/(site)/shop/page.tsx
  |
  | calls getCatalogueProductData()
  v
src/lib/catalogue-presets.ts
  |
  | uses getDb()
  v
src/db/index.ts
  |
  | Drizzle + Neon
  v
Neon PostgreSQL
```

This is used for public product display.

Why this is good:

Products are rendered on the server.
The browser receives a ready page.
No public /api/products call is needed.
DATABASE_URL remains server-side only.
SEO is better because products are already in the rendered page.

The page can be cached and revalidated.
Database load can be much lower under traffic.

Product Display Flow

Catertech public shop display:

```
User opens /shop
  |
  v
Next.js server page runs
  |
  v
getCatalogueProductData()
  |
  v
Drizzle query
  |
  v
Neon PostgreSQL
  |
  v
Product data mapped for UI
  |
  v
Rendered shop page cached by Next.js
  |
  v
User receives catalogue page
```

Important file:

```
src/app/(site)/shop/page.tsx
```

This page uses:

```
export const revalidate = 60;
```

Meaning:

The public shop page can be cached and refreshed approximately every 60 seconds.

This is better for public catalogue traffic than forcing every browser to call an API route.

Database Load Example

Assume 10,000 users visit /shop in 1 minute.

If using client API without cache

Flow:

```
10,000 users
  |
  v
10,000 browser requests to /api/products
  |
  v
10,000 API route executions
  |
  v
Potentially 10,000 Neon database queries
```

Result:

- More API traffic
- More database queries
- More server work
- Potentially higher hosting/database cost
- More loading states in browser
- Worse SEO if product content arrives only after client-side JavaScript

Catertech current better flow

Flow:

```
10,000 users visit /shop
      |
      v
Next.js serves cached rendered output for most visitors
      |
      v
Neon is queried much less often
```

Result:

- Lower database pressure
- Lower API overhead
- Better first page load
- Better SEO
- Lower maintenance complexity
- Usually lower running cost

This is why the current product display approach is a strong production choice.

When API Calls Are Still Required

API calls are still the right choice for these areas:

- Login
- Logout
- Signup OTP
- Contact form submit
- Trade enquiry submit
- RFQ submit
- Quote/cart submit
- Chatbot lead submit
- Admin create product
- Admin update product
- Admin delete product
- Admin user management
- Admin category management
- Admin product preset management
- Upload media

Reason:

These actions are user-specific, private, mutating, or need request validation. They should go through controlled backend API routes.

When Direct Drizzle Without API Is Better

Direct server-side Drizzle queries are better for:

- Public catalogue listing
- Public product detail pages
- Server-rendered admin page initial data where fresh server data is required

SEO pages that need database content before HTML is sent

Reason:

The query runs on the server.
Secrets stay on the server.
No extra internal HTTP request is needed.
Next.js can cache/revalidate public pages.
The browser gets ready-to-render content.

Implemented Security

The project has several important security items already implemented.

Admin Route Protection

Implemented in:

```
middleware.ts
src/lib/auth-user.ts
src/lib/admin-roles.ts
src/lib/user-auth-session.ts
```

What it does:

Protects /admin pages.
Protects /api/admin routes.
Requires signed-in staff session for admin access.
Supports admin and superadmin roles.
Restricts contacts and users areas to superadmin.
Redirects unauthenticated admin visitors to login.
Returns 401/403 for unauthorized admin API calls.

Signed HTTP-Only Session Cookie

Implemented in:

```
src/lib/user-auth-session.ts
```

What it does:

Creates HMAC-signed auth tokens.
Stores session in an HTTP-only cookie.
Uses SameSite=Lax.
Uses Secure cookies in production.
Verifies token expiry.
Uses timing-safe signature comparison.

Why this matters:

Client-side JavaScript cannot read HTTP-only cookies.
Signed cookies help prevent session tampering.
Secure cookies protect production traffic over HTTPS.

Password Hashing

Implemented in:

```
src/app/api/auth/login/route.ts
src/app/api/auth/signup/send-otp/route.ts
```


What it does:

Uses bcrypt password hashing.
Compares submitted password with stored password hash.
Does not store plain text passwords.

OTP Signup Flow

Implemented in:

src/app/api/auth/signup/send-otp/route.ts
src/app/api/auth/signup/verify-otp/route.ts
src/lib/auth-otp.ts

What it does:

Generates numeric OTP.
Stores hashed OTP challenge.
Uses expiry time.
Uses resend cooldown.
Requires AUTH_OTP_PEPPER in production.
Sends OTP by SMTP.

Rate Limiting

Implemented in:

middleware.ts
src/lib/security.ts
src/app/api/auth/login/route.ts

Current limits include:

/api/auth/login	20 requests per 15 minutes per IP
/api/auth/signup	12 requests per 15 minutes per IP
public form APIs	20 requests per 10 minutes per IP
/api/upload	80 requests per 10 minutes per IP
/api/admin	300 requests per minute per IP
login failures	blocked after repeated failed attempts

Note:

The current limiter is in-memory. This is useful, but for multi-region or many serverless instances, a shared rate limiter such as Redis/Upstash, Vercel Firewall, or provider-level WAF would be stronger.

Same-Origin Checks For Mutating APIs

Implemented in:

middleware.ts
src/lib/security.ts

What it does:

Checks Origin and Referer for POST, PUT, PATCH, DELETE API requests.
Rejects requests from unexpected origins.
Helps reduce CSRF-style abuse.

Note:

This is a good baseline. For very sensitive actions, explicit CSRF tokens can be added later.

Security Headers

Implemented in:

```
src/lib/security.ts
middleware.ts
```

Headers added:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=(), usb=()
Content-Security-Policy: base-uri 'self'; object-src 'none'; frame-ancestors 'none';
  form-action 'self'
```

Why this matters:

- Reduces clickjacking risk.
- Reduces MIME sniffing risk.
- Restricts unnecessary browser permissions.
- Limits dangerous embedded object usage.

Upload Security

Implemented in:

```
src/app/api/upload/route.ts
src/lib/media-storage.ts
```

What it does:

- Requires admin session for uploads.
- Allows images and MP4/WebM videos.
- Blocks SVG uploads.
- Limits multipart image upload size.
- Sanitizes uploaded filenames.
- Uses random UUID in storage keys.
- Uploads to configured media storage provider.

Why blocking SVG matters:

- SVG files can contain scripts or dangerous embedded content if served incorrectly.
- Blocking SVG reduces XSS risk for uploads.

Input Sanitization

Implemented in:

```
src/lib/security.ts
src/lib/sanitize.ts
```

Used by:

- Auth routes
- Public form routes
- Admin routes where applicable

What it does:

Normalizes/sanitizes email, text, phone, and multiline text inputs.
Reduces dirty data and common injection-style payloads before storage/use.

Secrets Stay Server-Side

Important environment variables:

DATABASE_URL
AUTH_SESSION_SECRET
SESSION_SIGNING_SECRET
AUTH_OTP_PEPPER
SMTP settings
media storage keys

These are used only in server-side files and API routes. They must not be exposed to client components.

Neon/Drizzle Production Notes

Catertech uses:

Neon PostgreSQL
Drizzle ORM
@neondatabase/serverless
drizzle-orm/neon-http

Important production guidance:

Use pooled Neon connection string for high traffic serverless deployments.
Enable autoscaling based on expected traffic.
Choose minimum compute size based on working set/cache needs.
Avoid querying Neon on every public page request when cached rendering is possible.
Use indexes for common product filters/order/search paths.
Disable scale-to-zero for production if cold-start latency is unacceptable.
Keep scale-to-zero enabled for lower-cost low-traffic environments if latency is acceptable.

Neon documentation notes:

Connection pooling helps avoid max connection pressure in serverless applications.
Autoscaling works best when the active working set can fit in the minimum compute cache.
Scale to zero reduces inactive compute cost, but wakeup adds latency.

Official references:

Next.js Server and Client Components:
<https://nextjs.org/docs/app/getting-started/server-and-client-components>

Next.js Backend for Frontend guide:
<https://nextjs.org/docs/app/guides/backend-for-frontend>

Next.js caching/revalidation:
<https://nextjs.org/docs/app/guides/caching-without-cache-components>

Neon connection pooling:
<https://neon.com/docs/connect/connection-pooling>

Neon compute/autoscaling:
<https://neon.com/docs/manage/endpoints/>

Neon scale to zero:
<https://neon.com/docs/introduction/scale-to-zero>

Recommended Future Improvements

The project is already structured reasonably for production. These are future improvements, not urgent blockers.

1. Add Server Service Layer

Recommended future structure:

```
src/server/  
  services/  
    product-service.ts  
    quote-service.ts  
    enquiry-service.ts  
    user-service.ts  
  
  repositories/  
    product-repository.ts  
    user-repository.ts  
    enquiry-repository.ts  
  
  validators/  
    product-validator.ts  
    quote-validator.ts
```

Benefit:

API routes become thinner.
Database logic becomes reusable.
Code is easier to test.
Large route files become easier to maintain.

2. Add Shared Distributed Rate Limiting

Current in-memory rate limiting is good as a baseline. For large production traffic, use a shared limiter.

Options:

Redis/Upstash
Vercel Firewall/WAF
Cloudflare WAF/rate limits
Database-backed rate counters if traffic is small

3. Add Product Query Pagination/Search At DB Level

If product count becomes large, avoid loading all products at once.

Better future flow:

Search/filter request -> DB WHERE/LIMIT/OFFSET or cursor pagination -> small response

4. Add On-Demand Revalidation After Admin Product Changes

When admin creates/updates/deletes a product, the app can revalidate:

```
/shop  
/shop/[slug]  
related category/product pages
```

This keeps public pages fast while making product edits visible quickly.

5. Add Stronger CSP If Needed

Current CSP is a good baseline. A stricter CSP can be added later after confirming all image, script, font, and media domains.

Final Architecture Summary

